Elliptic Curve Based Password Authenticated Key Exchange Protocols

Colin Boyd[1], Paul Montague[2], and Khanh Nguyen[2]

[1] Information Security Research Centre,
Queensland University of Technology,
Brisbane 4001, Australia
boyd@isrc.qut.edu.au
[2] Motorola Australia Software Centre,
2 Second Ave, Mawson Lakes, SA 5095, Australia
pmontagu@asc.corp.mot.com, knguyen@asc.corp.mot.com

Abstract. We investigate password authenticated key exchange (PAKE) protocols in low resource environments, such as smartcards or mobile devices. In such environments, particularly in the future, it may be that the cryptosystems available for signatures and/or encryptions will be based on elliptic curves, because of their well-known advantages with regard to processing and size constraints. As a result, any PAKE protocols which the device requires should also preferably be implemented over elliptic curves. We show that the direct elliptic curve (EC) analogs of some PAKE protocols are insecure against partition attacks. We go on to propose a new EC based PAKE protocol. A modified version of the protocol for highly constrained devices, such as smartcards, is also presented.

1 Introduction

A protocol that allows two parties to agree on a shared secret key is commonly known as a key exchange protocol. The protocol is said to be *authenticated* if the protocol authenticates one party to the other during the protocol run. Further, if the means of authentication is by a simple password known by both entities, the authenticated key exchange protocol is said to be *password-based*.

Recently, password authenticated key exchange (PAKE) has received significant interest from the research community. One of the reasons for this is that PAKE protocols can be used to establish an authenticated and secret channel between two parties without relying on the existence of a Public Key Infrastructure (PKI), and provides security against both active and off-line dictionary attacks. This is certainly appealing in many environments where the deployment of a PKI is not possible or would be overly complex.

The first PAKE protocol, known as Encrypted Key Exchange (EKE), was suggested by Bellovin and Merritt [3]. Subsequently many other PAKE protocols have been proposed. Nonetheless, most of the protocols have been proposed for only RSA or Discrete Logarithm (DL) settings so far. It seems that most authors

V. Varadharajan and Y. Mu (Eds.): ACISP 2001, LNCS 2119, pp. 487–501, 2001.

have presumed that the adaptation of DL based protocols to the elliptic curve (EC) environment is straightforward. To the best of our knowledge, no concrete EC based PAKE protocol has been proposed in the literature.

In a low resource environment, the natural choice for cryptographic protocols would be an EC implementation. This is due to the low computation and storage costs of EC based protocols. Since EC primitives may be the only ones available in certain environments, it is important to study the precise adaptation of DL-based PAKE protocols to an EC setting.

In this paper, we investigate EC analogs of PAKE protocols. We show that direct EC analogs of the EKE protocol and its variants are susceptible to partition attacks. We then go on to propose an EC encrypted key exchange protocol that is secure against partition attacks. We further propose a modification of the protocol for low resource (*e.g.* smartcard) applications. Here we stress that the EC analogs of other PAKE protocols such as SPEKE[5] or PAK[4] do not immediately appear to suffer from the partition attack described in this paper.

The remainder of this paper is organized as follows. Section 2 gives an introduction to PAKE. This includes a discussion on the possible attacks against a PAKE protocol. Section 3 gives a detailed description of the EKE protocol. A discussion on how a partition attack can be applied to the protocol is also given in this section. Section 4 reviews the concept of elliptic curves and twisted elliptic curves, establishing some notation and elementary results for the subsequent sections. Section 5 proposes a new elliptic curve encrypted key exchange protocol. In the section, we also justify our solution by showing that trivial solutions are insecure against partition attacks. A modification of the proposed protocol for smartcard applications is also included in the section. This has a DL analog, which is briefly described and discussed. Finally, conclusions are presented in Section 6.

2 Password Authenticated Key Exchange

In its simplest form, a PAKE protocol involves two parties both possessing the same secret, referred to as the password. This password is typically short, since it usually has to be memorised by a human participant. The PAKE protocol allows the involved parties to exchange information from which each party can derive a secret key. This secret key satisfies all requirements of a conventional Diffie-Hellman key exchange protocol. In particular, this shared secret key is not known to any parties not involved in the exchange protocol. Furthermore, the nature of password authentication ensures that a party can follow the protocol correctly (and thus be accepted by the other party) only if the party knows the correct password. During the protocol, a party can always detect whether the other party in the exchange possesses the correct password.

The first PAKE protocol was the EKE protocol proposed by Bellovin and Merritt [3]. The idea of their proposal is to use the password to symmetrically encrypt the protocol messages of a standard Diffie-Hellman key exchange. An attacker could decrypt the symmetric encryption by guessing the password but

could not tell whether the decryption results in a valid message. However, Patel subsequently showed [6, 7] that EKE protocols are susceptible to a partition attack (see below for further details). The solution to prevent the partition attack, is to carefully choose the system parameters so that the attack is no longer applicable. Fortunately, such restrictions on the system parameters do not introduce any performance penalty or security issues.

Another well-known PAKE protocol is Simple Password-authenticated Exponential Key Exchange (SPEKE) proposed by Jablon [5]. The idea of the protocol is to involve the password in computing the base used in conventional Diffie-Hellman key exchange. The protocol also introduces two extra steps for authenticating the generated secret key for each party respectively.

Recently two new PAKE protocols with provable security have been proposed. Bellare *et al.* [1] have given a specific variant of EKE which is provably secure in the ideal cipher model. The protocol of Boyko *et al.* [4] is also a variant of EKE in which the symmetric encryption takes the form of multiplication in the Diffie-Hellman group; it is provably secure in the random oracle model.

The security of a standard key exchange protocol may be measured against both *passive attacks* in which the adversary wiretaps valid instances of the key exchange protocol, and against *active attacks* in which the adversary may also masquerade as a valid protocol principal. Attacks may be aimed at obtaining information about the generated secret or about the long-term keying material.

The classical Diffie-Hellman key exchange and its EC analog are known to be secure against passive attacks. However, it is clearly insecure against an impersonation attack, because of the lack of an authentication mechanism being deployed. To prevent such attacks long-term keys are required, which in a PAKE protocol takes the form of the password. The password allows each party in the protocol to authenticate the other party and thus prevents an adversary from impersonating an authorized party in the PAKE protocol run.

However, the low entropy of a typical password opens a new possible attack against a PAKE protocol. This type of attack is often known as the off-line dictionary attack. In such an attack, the adversary tries to interact (even if unsuccessfully) with an honest party and also gathers information from exchanges between two honest parties. The adversary then applies a brute-force attack over the domain of the passwords (i.e. the dictionary) off-line. The attacker is successful if the gathered information can confirm which password in the dictionary is the valid one. A special class of the dictionary attack is the *partition attack*. In a partition attack, the adversary tries to use the gathered information to partition the password space (the dictionary) into feasible and infeasible passwords; if the latter set is large then the adversary may simply search through and eliminate all passwords in this set. Typically the correct password may be recovered after a number of valid sessions have been observed from the intersection of the feasible partition of the passwords for each session.

3 Diffie Hellman Encrypted Key Exchange

In this section, we give an overview of the Diffie-Hellman based encrypted key exchange (DH-EKE) protocol. This is one of the three variants of EKE proposed by Bellovin and Merritt in [3]. We also provide a brief description of how a partition attack can be applied against the protocol. Patel [6] gives further details.

3.1 The DH-EKE Protocol

The DH-EKE protocol involves two parties, Alice - the initiator of the protocol and Bob. Alice and Bob share a secret password P that is not known to any other party. There is also a publicly known prime number p and a publicly known generator g of the field $\mathsf{GF(p)}$. Also a symmetric encryption algorithm $Enc()$, its corresponding decryption algorithm $Dec()$ and a one-way hash function $\mathcal{H}()$ are publicly known. A variant of the DH-EKE protocol is as follows (see also figure

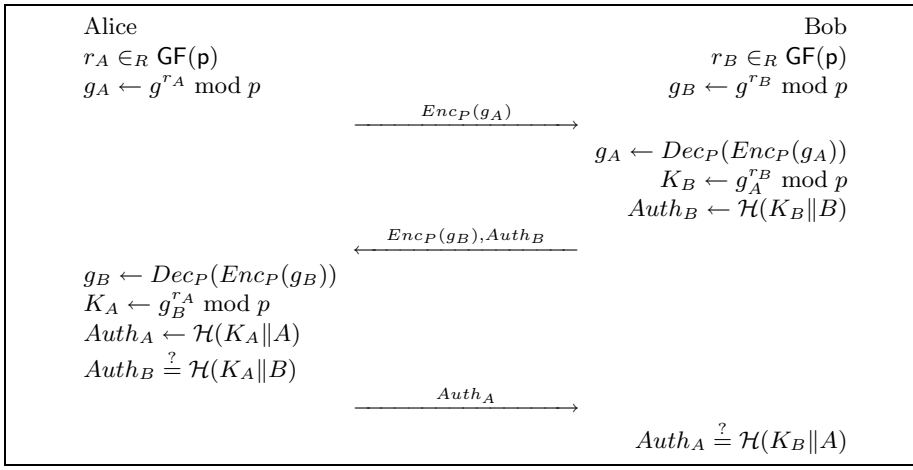

Fig. 1. A variant of the DH-EKE protocol

1):

1. Alice and Bob generate random numbers r_A and r_B and compute $g_A = g^{r_A} \bmod p$ and $g_B = g^{r_B} \bmod p$ respectively.
2. Alice and Bob encrypt g_A and g_B using the shared password P to generate $Enc_P(g_A)$ and $Enc_P(g_B)$ respectively.
3. Alice sends $Enc_P(g_A)$ to Bob.
4. Upon receiving $Enc_P(g_A)$, Bob computes $g_A = Dec_P(Enc_P(g_A))$. Bob then computes the key $K_B = g_A^{r_B}$. Bob then generates the authenticator $Auth_B = \mathcal{H}(K_B \| B)$ of K_B.

5. Bob sends $Enc_P(g_B)$ and $Auth_B$ to Alice.
6. Upon receiving $Enc_P(g_B)$ and $Auth_B$, Alice applies Dec_P to $Enc_P(g_B)$ to recover g_B. Alice then computes $K_A = g_B^{r_A}$ and verifies that $Auth_B \equiv \mathcal{H}(K_A \| B)$.
7. If the verification passes, Alice accepts $Auth_B$ and sends to Bob $Auth_A = \mathcal{H}(K_A \| A)$.
8. In turn, Bob checks that $Auth_A \equiv \mathcal{H}(K_B \| A)$. If the verification passes, Bob accepts $Auth_A$.

The protocol is successful if both Alice and Bob accept $Auth_B$ and $Auth_A$ respectively. The generated key then is $K = K_A = K_B$. The completeness is due to $K_A = g_B^{r_A} = g^{r_A r_B} = g_A^{r_B} = K_B$.

The protocol described here is not the exact description of the original DH-EKE. It is in fact an instance of the construction described by Bellare and Rogaway [2]. The difference between this protocol and the original DH-EKE protocol is the existence of values $Auth_A$ and $Auth_B$. These two values are to authenticate Alice to Bob and Bob to Alice respectively. In the original DH-EKE, a different authentication mechanism is used. Nonetheless both methods achieve the same goal.

3.2 Variants of the DH-EKE Protocol

The DH-EKE protocol can be modified in many ways. One type of variant changes the construction of the authentication part as shown above.

Another modification that can be made to the protocol is to omit the encryption with the password P by either Bob or Alice (but not both). Then instead of sending the encryption of g_A or g_B, Alice or Bob shall send the plaintext g_A or g_B to the other party. In this variant, the order of authentication between Alice and Bob must also be modified. The rule is that the party who does not perform any encryption in the first part, must initiate the authentication step. This prevents an active adversary using the authentication field to mount an off-line dictionary attack. Patel [6] discusses the security of such omissions in detail.

3.3 A Partition Attack against DH-EKE Family

Patel [7, 6] has shown that the DH-EKE protocol and its variants are susceptible to a partition attack if the values of g and p are not chosen carefully.

If the value g is not a generator of $\mathsf{GF(p)}$ but only a generator of a subgroup of order q over $\mathsf{GF(p)}$, an adversary can mount a partition attack as follows. Firstly the adversary obtains $Enc_P(g_A)$ by wiretapping an exchange between Alice and Bob. Next, the adversary tries to decrypt $Enc_P(g_A)$ using a password P_i. If the password P_i is correct, the decryption will result in a value g_A which is of order q. If P_i is not the correct password, it is likely that the decryption will result in a value g'_A which is not of order q. The probability that the decryption will result in a value of order r, $r|p-1$, for a random P_i is $\frac{\phi(r)}{p}$. The attacker can check, by

raising the result to the power q and checking whether 1 is obtained, whether the order divides q. It can then be seen that the probability that 1 is obtained, for an incorrect password, is $\frac{q}{p-1}$. Thus the possible space of valid passwords is reduced by a factor of $\frac{q}{p-1}$, on average, by observing one exchange session. Over a number of sessions the space of valid passwords will be narrowed down to a single password at a logarithmic rate.

To avoid the attack, it is suggested that g has to be a generator of $\mathsf{GF(p)}$ and that if Alice is allowed to choose p and g for the protocol, Bob must check that g is indeed a generator.

Similar partition attacks are possible if the value of p is not chosen carefully. In this case, if trial decryption of $Enc_P(g_B)$ with candidate passwords leads to values equal to or larger than p, then these candidate passwords may be eliminated.

4 Elliptic Curves and Twisted Elliptic Curves

In this section, we review the basic notation and definitions of elliptic curves, as well as the concept of the twist of an elliptic curve. This concept is crucial to the construction of our new protocol.

The use of elliptic curve groups in public key cryptography was first proposed by Koblitz [9] and Miller [8]. Recently there has been much focus on such cryptosystems, with adoption in various standards, such as WAP [10], IEEE P1363 [11] and ANSI X9.62 [12]. This is because public key methods based on elliptic curve groups typically have lower processing requirements, and can achieve the same level of security with considerably shorter key sizes than cryptosystems based on the more traditional RSA and standard discrete logarithm schemes. As such, elliptic curve cryptographic systems are ideal for environments where processing power, time and/or network bandwidth are at a premium. Consequently, there is a drive for the adoption of elliptic curve based protocols in wireless environments and smartcard based applications.

For the sake of simplicity, we consider the case of a curve in a field of characteristic greater than 3. For the case of characteristic two, analogous statements hold true [13]. A more general discussion of twists of elliptic curves is given by Silverman [14].

Following Blake *et al* [13], we consider a curve defined over the field $K = \mathsf{GF(q)}$, where $q = p^m$ and $p > 3$ is a prime. Consider the curve in short Weierstrass form, *i.e.*

$$E_{a,b} : Y^2 = X^3 + aX + b.$$

Set $g(X) = X^3 + aX + b$, so that the equation of the curve becomes $Y^2 = g(X)$. As shown in the literature, we may define an additive (abelian) group on the set of points on this curve (taken together with the point at infinity). It is this group, and the discrete logarithm problem defined therein, which may be used to define cryptographic primitives.

In the following sections, use shall be made of the concept of the twist of an elliptic curve. Consider then the curve $E_{a',b'}$ where $a' = v^2 a$ and $b' = v^3 b$ for

some $v \in K^*$. If we set $g_v(X) = v^3 g(X/v)$, then we have the equation of the curve $E_{a',b'}$ given by $Y^2 = g_v(X)$. However, $E_{A,B}$ is isomorphic to $E_{a,b}$ over K if and only if $A = u^4 a$ and $B = u^6 b$ for some $u \in K^*$. Hence the curve $E_{a',b'}$ is isomorphic to $E_{a,b}$ if v is a quadratic residue. Furthermore, if v is not a quadratic residue, then there is a unique such curve, up to isomorphism over K. This is called the **twist** of $E_{a,b}$. [Note that over $\mathsf{GF}(\mathsf{q}^2)$ the original curve and its twist are isomorphic.]

An observation (see *e.g* [13]) that will be of use in the subsequent sections is the following. Consider $X \in K$ for which $g(X) \neq 0$. Then if $g(X)$ is a non-zero quadratic residue, X is the x-coordinate of a point on $E_{a,b}$. Otherwise, $g_v(vX)$ is a quadratic residue, and hence vX is the x-coordinate of a point on $E_{a',b'}$. The following lemma summarises the relevant properties about the connection between a curve and its twist.

Lemma 1. *Let*

$$C = \{X \in K | g(X) \text{ is a quadratic residue in } K\}$$
$$T = \{X \in K | g_v(vX) \text{ is a quadratic residue in } K\}$$

Then

1. $C \cap T = \emptyset$. *If* $g(X) \neq 0$ *then* $X \in C \cup T$.
2. $X \in C \iff g(X) \neq 0$ *and* $\exists Y$ *such that* $(X, Y) \in E_{a,b}$.
3. $X/v \in T \iff g(X/v) \neq 0$ *and* $\exists Y$ *such that* $(X, Y) \in E_{a',b'}$.

Note also in the following sections that we will need to represent the points of the elliptic curve in a compressed form. Denoting the points naively, an affine point (X, Y) requires $2n$ bits, where n is the bit length of the underlying field. There is a trivial reduction to $n+1$ bits by observing that, given X, the value of Y is one of the two solutions of a quadratic equation. A single bit may be used to distinguish between these two solutions. While this compressed form is clearly convenient and cost effective (particularly in situations in which transmission bandwidth is limited) in the following we shall see that it is essential in order to obviate simple attacks on the protocols described.

In order for the elliptic curve $E_{a,b}$ to be suitable for use in a cryptosystem, it is required that [13]:

- the group has a subgroup of large prime order,
- the curve is not anomalous ($q = N_{a,b} = p$, where $N_{a,b}$ is the group order),
- the curve satisfies the MOV condition [13, 11] (the smallest value of l such that $q^l = 1 \bmod N_{a,b}$ should be large).

In the protocols we will describe in which both the curve and its twist are used, these properties are also required of the twist. [The order of the group of the twisted curve is given by the relation $N_{a,b} + N_{a',b'} = 2q+2$, in the case that $E_{a',b'}$ is the twist of $E_{a,b}$. Hence there is little additional effort required in verifying these properties hold for any generated curve and its twist over and above for the curve alone.] We shall assume in the following that the elliptic curves we use satisfy these security constraints.

5 Elliptic Curve Encrypted Key Exchange

In this section, we present our elliptic curve encrypted key exchange (EC-EKE) protocol. We further show an unbalanced variant of the protocol. Such a protocol may be utilized in a situation in which one of the parties has limited processing power, such as communication between a smartcard or mobile device and a terminal or server. First, however, we will show that the direct EC analog of DH-EKE protocol is insecure and thus justify our design.

Throughout this section, we shall use the notation of the preceding section, and consider a curve $E_{a,b}$, together with its twist $E_{a',b'}$, for suitably chosen a' and b'. For further simplicity, we restrict to the case $q = p$, *i.e.* consider the curve over $\mathsf{GF}(\mathsf{p})$.

5.1 Trivial Protocols Are Insecure against Partition Attacks

Following the usual methodology of replacing the DL group operations with operations in an EC group, the obvious procedure would be to design the EC-EKE protocol as the direct EC analog of a variant of the DH-EKE protocol.

The principle of any variant of the DH-EKE protocol is that Alice, say, will encrypt and send the encryption of $g_A = g^{r_A}$ to Bob. The direct analog in EC is that Alice will encrypt the point $G_A = r_A * G$ and send the encryption $Enc_P(G_A)$ to Bob.

The trivial encryption method is to encrypt (X_A, Y_A) in $Enc_P(G_A)$, where $G_A = (X_A, Y_A)$. In this case, the adversary can simply apply an off-line dictionary attack for a valid $Enc_P(G_A)$ by decrypting $Enc_P(G_A)$ with every password P_i in the password space. Clearly, if P_i is incorrect, the decryption should result in a random pair (X_i, Y_i). Even if X_i, $Y_i \in \mathsf{GF}(\mathsf{p})$, the point (X_i, Y_i) is on the elliptic curve $E_{a,b}$ only if it satisfies

$$Y_i^2 = g(X_i) \bmod p.$$

This happens with a probability of order $1/p$ for a random pair $(X_i, Y_i) \in \mathsf{GF}(\mathsf{p})^2$. Typically the size of the password space is much less than p. Hence the adversary should be able to identify the correct password P given a single valid encryption $Enc_P(G_A)$ using such a dictionary attack.

As discussed in the previous section, an alternative more compact form for the representation of, G_A of the point is its compressed form, in which the y-coordinate is replaced by a single bit. If the adversary applies a dictionary attack on the encryption $Enc_P(G_A)$ in this situation, the adversary will be able to recover the x-coordinate X_i for the password choice P_i and a bit indicating which solution Y_i to choose. If the password P_i is incorrect, X_i will be essentially random. Observe however that a random X_i is a valid x-coordinate only if $g(X_i)$ is a quadratic residue. ¿From Hasse's theorem (see *e.g.* [13]), we know that the number of such values X_i in $\mathsf{GF}(\mathsf{p})$ is in the range $[(p+1)/2-\sqrt{p}, (p+1)/2+\sqrt{p}]$. Thus a random X_i in $\mathsf{GF}(\mathsf{p})$ is a valid x-coordinate of $E_{a,b}$ with a probability in the range $[1/2-O(1/\sqrt{p}), 1/2+O(1/\sqrt{p})]$. Hence the adversary can successfully

apply a partition attack by reducing the possible password space by roughly half given a valid $Enc_P(G_A)$. This means the password can be recovered given a number of sessions of the order of the log of the size of the password space.

5.2 An Elliptic Curve Encrypted Key Exchange Secure against Partition Attacks

In the previous subsection, we have shown that the direct EC analog of any variant of the DH-EKE protocol is insecure. In this subsection, we propose a new EC-EKE protocol and show that the protocol is secure against partition attacks.

In order to avoid the elementary attacks described in the previous subsection, the simplest approach would be to ensure that any candidate x-coordinate observed by an adversary is valid. This would then obviate the partition attack.

We recall from Lemma 1 that for $X \in \mathsf{GF}(\mathsf{p})$ for which $g(X) \neq 0$, then if $g(X)$ is a non-zero quadratic residue, X is the x-coordinate of a point on the curve. Otherwise, $g_v(vX)$ is a quadratic residue, and hence vX is the x-coordinate of a point on the twist of the curve. Using this observation, an EC-EKE protocol is designed as follows.

Let $E_{a',b'}$ be a twisted curve of $E_{a,b}$. Let G be a generator point of the curve $E_{a,b}$ and H be a point which generates the curve $E_{a',b'}$. Here we assume that the points in $E_{a,b}$ and $E_{a',b'}$ respectively form a cyclic group. There are two important remarks that should be made in relation to the choice of these parameters.

- Generation of suitable curve/twist pairs will inevitably take considerably longer than when the properties of the twist are irrelevant. However, this is a one time setup cost and a single curve may be re-used for a large number of different users.
- It is common to run Diffie-Hellman exchange in prime order subgroups in order to avoid small subgroup attacks. To preserve the properties which prevent partition attacks we have to use generators of the whole of the curve groups. Therefore we need to additionally require *either* that the curve group and its twist have prime order, *or* (more practically) check that all received values are not in any small subgroup. The latter may be achieved by making a number of simple checks depending on the factorisation of the curve cofactor. We regard this matter as an implementation detail and therefore ignore it in the protocol description.

To generate a password-authenticated shared secret key K, Alice and Bob proceed as follows (see figure 2):

1. Alice randomly selects either the curve $E_{a,b}$ or $E_{a',b'}$ for use in this run of the protocol.
2. Alice chooses a random r_A and computes $G_A = r_A * G$ if the selected curve is $E_{a,b}$ or $H_A = r_A * H$ otherwise.

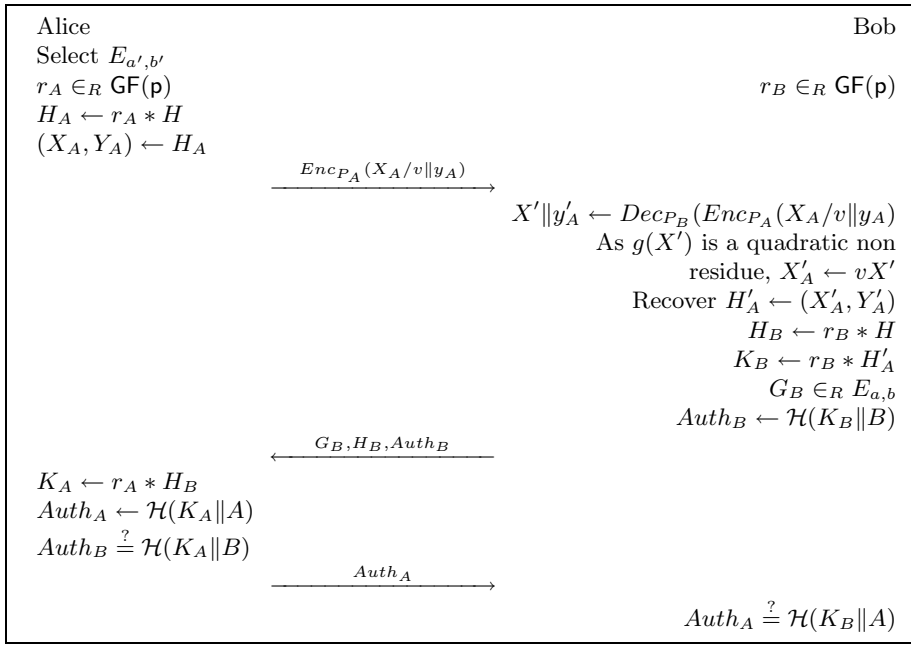

Fig. 2. The EC-EKE protocol: the chosen curve is the twisted curve $E_{a',b'}$

3. Alice compresses the point $G_A = (X_A, Y_A)$ (or $H_A = (X_A, Y_A)$) to (X_A, y_A) where y_A is a single bit representing Y_A in the compressed form.
4. If the (untwisted) curve $E_{a,b}$ is chosen, Alice sends $Enc_{P_A}(X_A\|y_A)$ to Bob. Otherwise Alice sends $Enc_{P_A}(X_A/v\|y_A)$ to Bob.
5. Upon receipt of the ciphertext, Bob decrypts it to obtain $X'\|y'_A$. If Bob finds that $g(X')$ is a quadratic residue, then Alice's chosen curve is the untwisted curve and the x-coordinate is $X'_A = X'$. Otherwise, Alice's chosen curve is the twisted curve and the x-coordinate is $X'_A = vX'$.
6. Once Bob has determined the x-coordinate X'_A, Bob recovers the point G'_A (or H'_A) from X'_A and y_A. Bob also chooses a random value r_B. Bob then computes $G_B = r_B * G$ and $K_B = r_B * G_A$ if the untwisted curve is chosen by Alice and chooses a random point H_B on the twisted curve. Otherwise Bob computes the points $H_B = r_B * H$ and $K_B = r_B * H_A$, and chooses a random point G_B on the untwisted curve.
7. Next Bob sends the points G_B and H_B and the authenticator $Auth_B = \mathcal{H}(K_B\|B)$ to Alice.
8. In turn, Alice computes $K_A = r_A * G_B$ if the untwisted curve $E_{a,b}$ is chosen. Otherwise Alice computes $K_A = r_A * H_B$.
9. Alice then verifies $Auth_B \equiv \mathcal{H}(K_A\|B)$. If so, Alice sends $Auth_A = \mathcal{H}(K_A\|A)$ to Bob.
10. Finally, Bob verifies that $Auth_A \equiv \mathcal{H}(K_B\|A)$. If so, the protocol is completed.

The shared secret key K is derived from K_A or K_B using a publicly known algorithm. As this step is not important in our protocol, we omit it here and simply assume that K_A and K_B are Alice's and Bob's copy of the shared secret key respectively.

The completeness of the protocol is as follows (assuming Alice chooses the untwisted curve - clearly a similar result holds in the other case). If $P_A = P_B$, at step 4, Bob will recover $X'_A = X_A$ and thus can determine that Alice chose the untwisted curve at step 1. This means that at step 6, Bob will recover the point $G'_A = G_A$ that is chosen by Alice. Thus we have $K_B = r_B * G'_A = r_B * r_A * G = r_A * G_B = K_A$ and the checks

$$Auth_B = \mathcal{H}(K_B \| B) \equiv \mathcal{H}(K_A \| B)$$

and

$$Auth_A = \mathcal{H}(K_A \| A) \equiv \mathcal{H}(K_A \| A)$$

follow.

5.3 Security

Formally proving the security for this protocol is difficult as this uses a symmetric encryption scheme for which no formal proof model exists. Under passive attacks, the security of this protocol is based on the fact that both $E_{a,b}$ and $E_{a',b'}$ are cyclic and G and H generate $E_{a,b}$ and $E_{a',b'}$ respectively. Thus the triplet $\{G_A, G_B, K\}$ (or $\{H_A, H_B, K\}$) is indistinguishable from a random set in $E_{a,b}$ (or $E_{a',b'}$). This implies that passive attacks are not feasible assuming that $\mathcal{H}()$ leaks no information to the attacker. For active attacks, we consider the protocol under three different types of attacks, namely impersonation attacks, off-line dictionary attacks and partition attacks.

For the impersonation attack:

- If the attacker impersonates Bob, the attacker will need to supply a valid $Auth_B$ to Alice given $Enc_{P_A}(X_A \| y_A)$. Alice will accept $Auth_B$ only if $Auth_B \equiv \mathcal{H}(K_A \| B)$. This requires Bob to know K_A when Bob computes $Auth_B$. This is possible only if Bob can derive the correct point G_A from the encryption $Enc_{P_A}(X_A \| y_A)$. This happens only if Bob knows the correct password.
- If the attacker impersonates Alice, the attacker will have to give Bob the value $Enc_{P_i}(X_A \| y_A)$ for a password P_i of the attacker's choice. Unless $P_i = P_A$, the point G'_A or H'_A that Bob recovers will differ from the point G_A (or H_A) that the attacker has chosen in the first place. Bob accepts the protocol only if Bob accepts $Auth_A$. This happens only if $K_A = K_B$ or that the attacker is able to compute K_B. Assuming that $\mathcal{H}()$ is a random oracle, the attacker will have to compute $K_B = r_B * G'_A$ (or $r_B * H'_A$) from G_A and G_B (or H_A and H_B). This is solvable only if the attacker can solve the EC discrete logarithm problem for G'_A and G_A (or H'_A and H_A).

For dictionary and partition attacks, an attacker can decrypt $Enc_{P_A}(X\|y)$ using a password P_i to obtain $(X_i\|y_i)$ where X_i is random. However, as we have seen above, all $X_i \in \mathsf{GF}(\mathsf{p})$ are valid, with the X where $g(X) = 0$ being sufficiently negligible in practise. Furthermore as $E_{a,b}$ ($E_{a',b'}$) is cyclic, the point G_i (or H_i) is indistinguishable from a random point under the EC Diffie-Hellman assumption. Thus the decryption gives no useful information about the plaintext to the attacker provided the value of p is chosen suitably, exactly as for DH-EKE as discussed in section 3.3 and by Patel [7]. Note that it is essential in this protocol that the points be represented in compressed form prior to encryption. If not, then sufficient information will remain to perform a parition attack.

5.4 An Unbalanced Variant for Smartcards

As we have discussed, the use of EC methods is often advantageous in environments in which there are limited resources for computation. In severely constrained devices, such as smartcards, further optimizations may be required. In this subsection, we describe a suitable protocol for this case.

An example of the sort of situation we have in mind is that a user is required to authenticate to a smartcard by means of a password. The link between the user's password entry device (a terminal) and the smartcard may be insecure. A PAKE protocol is an ideal solution for this particular problem. In this scenario, authentication of the card back to the user is not required. However, it may easily be added to our proposed protocol if required.

The idea of the protocol is to optimize the number of scalar (EC) multiplications that the card has to perform, since this is the operation which is computationally expensive. This is achieved by fixing the value r_A generated by the card for all transactions. The price we pay is the loss of forward secrecy. However if we link this value to the secret stored in the card, the use of which the user authentication is there to protect, then if r_A is compromised the secret in the card is compromised, and thus there is no longer anything to protect. This means that forward secrecy is no longer a significant requirement.

The protocol is as follows. Again, we use the notation of section 4. To set up the protocol, the card chooses either $E_{a,b}$ or $E_{a',b'}$. For the sake of simplicity, let us assume that the card chooses $E_{a,b}$. The card then chooses a random value r_A, preferably linked to the secret stored in the card. It then computes $G_A = r_A * G$. The above process is performed only once. The card then stores r_A and G_A for future use. It also stores a counter c initially set to 0. When a user requires to establish a password-authenticated secret session with the card, the card and the user perform the following variant of the EC-EKE protocol:

1. The card increases the counter $c = c+1$ and sends to the user $Enc_{P_A}(X_A\|y_A)$ where X_A is the x-coordinate of G_A and y_A is the bit representing the y-coordinate Y_A of G_A.
2. The user decrypts $Enc_{P_A}(X_A\|y_A)$ to obtain X'. The user then determines whether $g(X')$ is quadratic residue and thus determines which is the chosen curve. In the case here, the user determines that $E_{a,b}$ is the chosen curve. Then the user constructs the point G'_A from $X'_A = X'$ and y_A.

3. Next the user chooses a random value r_B and computes $G_B = r_B * G$ and $K_B = r_B * G'_A$. Also the user chooses a random point H_B of $E_{a',b'}$. The user also constructs the authenticator $Auth = \mathcal{H}(K_B \| c)$. Then the user sends $Auth$, G_B and H_B to the card.
4. The card computes $K_A = r_A * G_B$ and verifies that $Auth \equiv \mathcal{H}(K_A \| c)$. If so, the card accepts the user and the protocol is completed.

The advantage of this scheme is that the card only needs to perform a single scalar multiplication, a saving of one scalar multiplication compared with the original protocol. This reduces the computational requirement for the card by about a half. There is a potential replay attack on the protocol if an old K_A or K_B value is available to an attacker; this is discussed further below.

It is clear that this protocol is secure against partition and dictionary attacks. A similar argument as for our EC-EKE protocol can be applied here. Impersonating the user would be as difficult as for the original protocol. This is because the tasks that the user has to perform and the card has to verify in regard to the user's information remain unchanged. The value c is introduced to compensate for the fixing of r_A. Thus, each session is different and a straight replay attack is not possible.

5.5 Some Comments on the DL Analog

For completeness, we make a few points regarding the DL analog of the above unbalanced scheme. The relevance is that the following discussion also touches on points of difference between the EC and DL schemes, showing again that the map between protocols is not always straightforward.

There is a naive analog of the above EC protocol as follows.

Let us suppose that we are working over the field $\mathsf{GF}(\mathsf{p})$ with g a generator. As before, the counter c is initialized to zero. In addition, the card chooses a random value r_A, preferably linked to the secret stored in the card. It then computes $g_A = g^{r_A} \bmod p$, and stores r_A and g_A for future use.

The protocol may then be as follows:

1. The card increases the counter $c = c + 1$ and sends to the user $Enc_{P_A}(g_A)$.
2. The user decrypts this to obtain g_A.
3. Next the user chooses a random value r_B and computes $g_B = g^{r_B} \bmod p$ and $K_B = {g'_A}^{r_B} \bmod p$. The user also constructs the authenticator $Auth = \mathcal{H}(K_B \| c)$. Then the user sends $Auth$ and g_B to the card.
4. The card computes $K_A = {g_B}^{r_A} \bmod p$ and verifies that $Auth \equiv \mathcal{H}(K_A \| c)$. If so the card accepts the user and the protocol is completed.

This naive protocol satisfies the same security properties as for the EC case. However, we note that compromise of K_A or K_B would allow an adversary to impersonate the user in subsequent protocol runs by replaying the compromised session. Though these values are temporary (the session key should be derived from them and then they should be deleted) and this vulnerability is therefore not of major significance, it is easily fixed by the following version of the protocol:

1. The card increases the counter $c = c + 1$ and sends to the user $Enc_{P_A}(g_A)$.
2. The user decrypts this to obtain g_A.
3. Next the user chooses a random value r_B and computes $g_B = g^{r_B} \bmod p$ and $K_B = {g'_A}^{r_B} \bmod p$. The user also constructs the authenticator $Auth = \mathcal{H}(K_B)$. Then the user sends $Auth$ and $Enc_{P_B}(c + g_B)$ to the card.
4. The card recovers g_B and computes $K_A = {g_B}^{r_A} \bmod p$ and verifies that $Auth \equiv \mathcal{H}(K_A)$. If so, the card accepts the user and the protocol is completed.

Note that the tying of the counter c to the user's DH value g_B by the password P_B prevents the sort of replay attack noted above should K_A or K_B be compromised. Of course, compromise of r_A is still fatal as discussed above.

It is interesting to note however that there does not seem to be an EC analog of the above protocol. The encryption on the data sent by the user could be mapped, as we have done above, to a protocol using a curve and its twist. However, the card would also have to use the same protocol ideas, and the user and card cannot make an independent choice of curve to use, *i.e.* if the card chooses the twisted curve then so must the user. A partition attack therefore becomes feasible again. We omit the details, but simply note it as an example of the distinction between DL and EC schemes which is not immediately apparent from a naive approach.

6 Conclusion

Motivated both by the recent interest in PAKE protocols and by the uptake of EC cryptographic schemes, we have considered the transition of the DL based PAKE protocol DH-EKE to the EC environment. We have demonstrated that the naive EC analogs of such schemes are vulnerable to partition attacks. Furthermore, we have proposed a secure EC variant, using the concept of the twist of an elliptic curve in order to render the effectiveness of the partition attack negligible. Unbalanced schemes, in both the DL and EC settings, adapted to severely resource limited participants on one side of the protocol, have also been proposed and examined. It is observed throughout that the transition to EC schemes cannot be made naively, and it is suggested that distinct protocols for the DL and EC environments may need to be considered.

Further development of these ideas is the subject of ongoing work. In particular, given that a curve and its twist over $\mathsf{GF}(\mathsf{p})$ are isomorphic over $\mathsf{GF}(\mathsf{p}^2)$, it might seem that the problems touched on in section 5.5 regarding introducing encryption of the data sent by both parties may be resolved if the two parties are able to derive the shared secret elliptic curve point in this larger structure.

Note that though PAKE schemes such as SPEKE [5] and PAK [4] do not immediately appear to suffer from the problems encountered above in transitioning to an EC analog, further investigation is required to clarify this statement. We have at the very least demonstrated that the development of EC analogs of PAKE protocols can be non-trivial.

Acknowledgements

This research is part of an ARC SPIRT project undertaken jointly by Queensland University of Technology and Motorola.

References

[1] M. Bellare, D. Pointcheval, and P. Rogaway. Authenticated key exchange secure against dictionary attacks. In *Advances in Cryptology – Eurocrypt'2000*, pages 139–155. LNCS vol. 1807, Springer-Verlag, 2000.

[2] Mihir Bellare and Phillip Rogaway. The AuthA protocol for password-based authenticated key exchange. In *Submission to IEEE P1363 study group*, 2000.

[3] S. Bellovin and M. Merritt. Encrypted key exchange: Password based protocol secure against dictionary attackes. In *Proceedings of the Symposium on Security and Privacy*, pages 72–84. IEEE, 1992.

[4] V. Boyko, P. MacKenzie, and S. Patel. Provably secure password-authenticated key exchange. In *Advances in Cryptology – Eurocrypt'2000*, pages 156–171. LNCS vol.1807, Springer-Verlag, 2000.

[5] D. Jablon. Strong password-only authenticated key exchange. *ACM Computer Communication Review*, 26(5):5–20, 1996.

[6] S. Patel. Number theoretic attacks on secure password schemes. In *Proceedings of the Symposium on Security and Privacy*, pages 236–247. IEEE, 1997.

[7] S. Patel. Information Leakege in Encrypted Key Exchange. In *Proceedings of the DIMACS Workshop on Network Threats*, 1997.

[8] V. Miller. Use of elliptic curves in cryptography. In *Advances in Cryptology – Crypto 85*, pages 417–426. LNCS vol.218, Springer-Verlag, 1986.

[9] N. Koblitz. Elliptic curve cryptosystems. *Math. Comp.*, 48:203–209, 1987.

[10] *Wireless Application Protocol – Wireless Transport Layer Security Specification*, Wireless Application Forum Ltd, 2000.

[11] *IEEE P1363 Study Group for Future Public-Key Cryptography Standards.* http://grouper.ieee.org/groups/1363/StudyGroup.

[12] *Public Key Cryptography for the Financial Services Industry: The Elliptic Curve Digital Signature Algorithm (ECDSA).* American National Standard for Financial Services, X9.62, 1998.

[13] I.F. Blake, G. Seroussi and N.P. Smart. *Elliptic Curves in Cryptography.* LMS Lecture Note Series 265, CUP, 1999.

[14] J.H. Silverman. *The Arithmetic of Elliptic Curves.* Springer-Verlag, GTM 106, 1986.